

PI-Safepay Cards—Tokenization

Document Number	OI - PI - 20260304
Document status	COMPLETED
Document owner	@Shayan Rasheed @Rizwan Zaffar
Solutions Architect	@Maqsood Ali @Owais Khalid @Saqlain Raza
Project Manager	TBD
Priority	TBD
Region	Pakistan
Operator	Safepay-Cards-Tokenization
Product	PAYIN

1. Overview

Safepay Cards-Tokenization is the card payment and vault management product offered by Safepay. It enables merchants to accept card payments processed via CyberSource, perform 3D Secure (3DS) authentication through the Safepay Atoms frontend SDK, and tokenize customer card details into reusable payment method tokens (`pm_` tokens) stored in the Safepay vault.

Simpaisa operates as the integrating payment aggregator in this model. Simpaisa's backend engineers call the Safepay APIs on behalf of downstream merchants. All requests are authenticated using a per-merchant secret key (`X-SFPY-MERCHANT-SECRET`) issued by Safepay.

This integration covers four payment and tokenization flows:

- **Flow 1 — Tokenization + Payment:** Collects raw card details from the customer, processes a live payment, and simultaneously saves the card as a reusable `pm_` token. The result is both a completed payment and a saved payment method attached to the customer profile.
- **Flow 2 — Token Initiated Payment (With 3DS):** Charges a customer using a previously saved `pm_` token with `entry_mode: "tms"` . 3DS authentication is performed via the Atoms module. The card capture step in Atoms is skipped — only the Payer Authentication component is used.
- **Flow 3 — Token Initiated Payment (Without 3DS):** Charges a customer using a saved `pm_` token in `mode: "unscheduled_cof"` (unscheduled card-on-file). No 3DS challenge and no Atoms module is required.
- **Flow 4 — Tokenization Without Charge:** Saves a customer's card without charging them using a zero-amount authorization (`is_account_verification: true`). After the card is tokenized and the `pm_` token is retrieved.

All four flows share the same authentication mechanism and base URL setup described in Sections 2 and 3.

2. Environments & Base URLs

All endpoint paths in this document are relative to the appropriate base URL. Replace `{{baseUrl}}` with the correct environment URL in all cURL examples.

Environment	Base URL	Notes
Sandbox	<code><https://sandbox.api.getsafepay.com ></code>	For integration testing. Use for all API calls except Payer Authentication Setup.
Production	<code><https://api.getsafepay.com ></code>	Live traffic. Real money movement.

3. Authentication

Every API request must include the merchant secret key as a request header:

```
1 X-SFPY-MERCHANT-SECRET:  
  {{merchant_secret_key}}/5579d044ed44a03ad91b9aa216fcec8c2c2b67f55247c18  
  bc13062ca5db36c8c
```

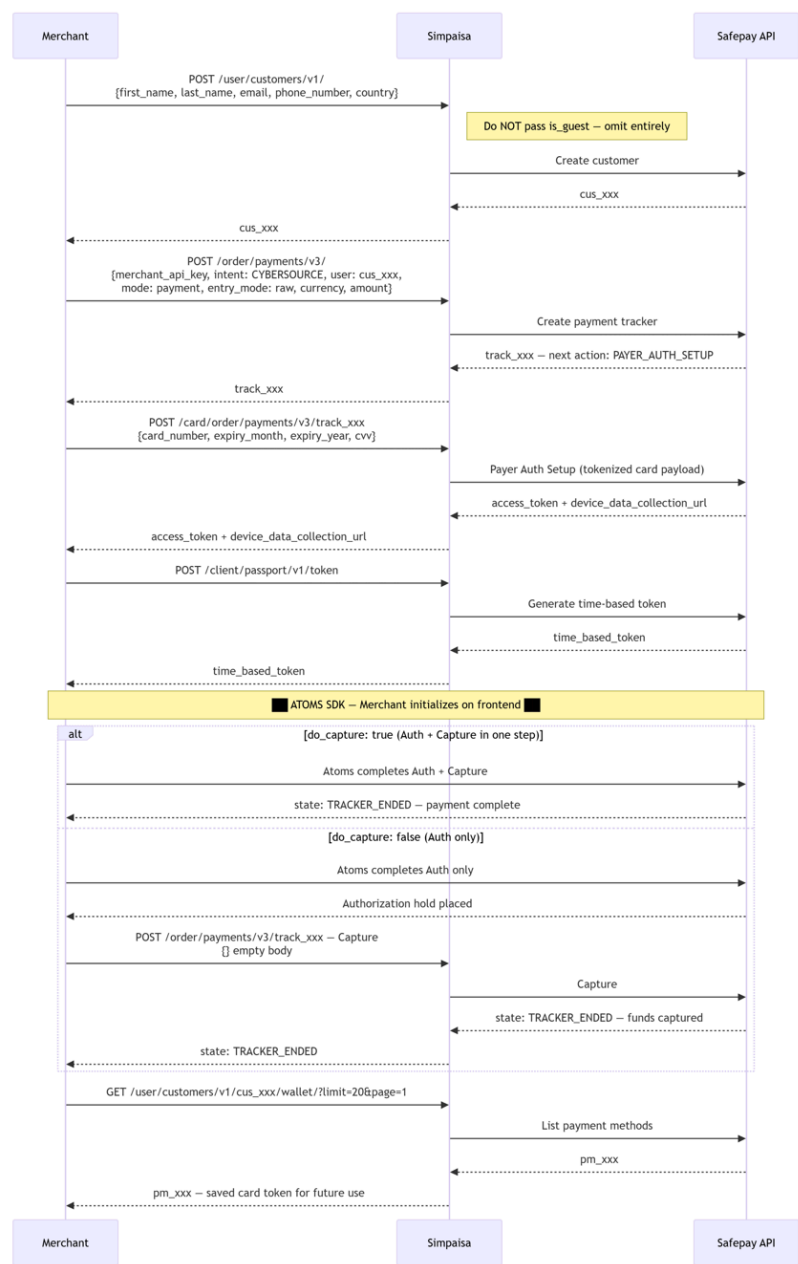
The `merchant_secret_key`: `5579d044ed44a03ad91b9aa216fcec8c2c2b67f55247c18bc13062ca5db36c8c` is issued by Safepay per merchant and authenticates all operations under that merchant identity.

Security Requirement: Store `merchant_secret_key` exclusively in a server-side secret manager (e.g., AWS Secrets Manager, HashiCorp Vault). Never embed it in frontend code, mobile applications, or version control. Exposure of this key compromises the entire merchant account.

4. Flow 1 — Tokenization + Payment

This flow collects raw card details from the customer for the first time, processes a live payment, and simultaneously tokenizes the card for future use. The result is both a successful payment and a saved payment method (`pm_` token) attached to the customer profile. The `entry_mode` for this flow is `"raw"`.

Sequence Diagram



Step 1. Create Customer

Create Customer

Description: Creates a new customer profile in the Safepay vault. Returns a `cus_` token that must be passed as the `user` parameter in the Payment API (Step 2) and used to retrieve saved payment methods later. **Do not pass** `is_guest: true` in this flow — omit the parameter entirely; including it will prevent proper tokenization against a persistent customer profile.

Method: POST

Endpoint: `/user/customers/v1/`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
first_name	Body	String	M	"Shayan"	Customer's first name.
last_name	Body	String	M	"Ahmed"	Customer's last name.
email	Body	String	M	"testing@simpaisa.com"	Customer's email address.
phone_number	Body	String	M	" +923370663196"	Customer's phone number in E.164 format.
country	Body	String	M	"PK"	ISO 3166-1 alpha-2 country code.
is_guest	Body	Boolean	O	—	Optional flag for guest checkout. Must NOT be passed in the tokenization flow. Omit this parameter entirely to ensure the card is saved against a persistent customer profile.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/user/customers/v1/" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4   --header "Content-Type: application/json" \  
5   --data '{  
6     "first_name": "Shayan",  
7     "last_name": "Ahmed",  
8     "email": "testing@simpaisa.com",  
9     "phone_number": "+923370663196",  
10    "country": "PK"  
11  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.token	String	M	"cus_62c5d7e8-7d80-43db-8385-	Customer token. Use this as the user

			c49872a23cb7"	value in Step 2.
data.merchant_api_key	String	M	"sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19"	Merchant API key linked to this customer.
data.email	String	M	"testing@simpaisa.com"	Customer email confirmed.
data.first_name	String	M	"Shayan"	Customer first name confirmed.
data.last_name	String	M	"Ahmed"	Customer last name confirmed.
data.phone_number	String	M	"923370663196"	Customer phone number confirmed.
data.country	String	M	"PK"	Country code confirmed.
data.is_guest	Boolean	M	true	Will appear in response even though not sent in request.
data.is_deleted	Boolean	M	false	Whether the customer profile is deleted.
data.created_at.seconds	Number	M	1772636744	Unix timestamp of profile creation.
data.updated_at.seconds	Number	M	1772636744	Unix timestamp of last profile update.
status.errors	Array	M	[]	List of errors. Empty on success.
status.message	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1  {
2    "data": {
3      "token": "cus_62c5d7e8-7d80-43db-8385-c49872a23cb7",
4      "merchant_api_key": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
5      "email": "testing@simpaisa.com",
6      "first_name": "Shayan",
7      "last_name": "Ahmed",
8      "phone_number": "923370663196",
9      "country": "PK",
10     "is_deleted": false,
11     "created_at": {
12       "seconds": 1772636744,
13       "nanos": 693509973
14     },
15     "updated_at": {
16       "seconds": 1772636744,
17       "nanos": 693510623
18     }
19   },
20   "status": {
21     "errors": [],
22     "message": "success"
23   }
24 }
```

Step 2. Create Payment Tracker (Payment API)

Payment API

Description: Creates a payment tracker — the central transaction object that tracks all state transitions for this payment. Returns a `tracker.token` (`track_` prefix) that is passed as a path parameter to all subsequent steps. For Flow 1, set `entry_mode: "raw"` and `mode: "payment"`. The `user` field must be the `cus_` token from Step 1.

Method: POST

Endpoint: `/order/payments/v3/`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
merchant_api_key	Body	String	M	"sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19"	Merchant API key (the <code>sec_</code> token).
intent	Body	String	M	"CYBERSOURCE"	Payment processor. Always <code>"CYBERSOURCE"</code> for card payments.
user	Body	String	M	"cus_62c5d7e8-7d80-43db-8385-c49872a23cb7"	Customer token from Step 1.
mode	Body	String	M	"payment"	Transaction mode. Set to <code>"payment"</code> for Flow 1.
currency	Body	String	M	"PKR"	ISO 4217 currency code.
amount	Body	Number	M	10000	Transaction amount in the smallest currency unit (paisas).
entry_mode	Body	String	M	"raw"	Card entry mode. Set to

					"raw" for direct card input in Flow 1.
metadata.source	Body	String	O	"xxxx"	Merchant-defined source identifier.
metadata.order_id	Body	String	O	"xxxx"	Merchant-defined order identifier for reconciliation.

Sample Request (cURL)

```

1 curl --request POST \
2   --url "{{baseUrl}}/order/payments/v3/" \
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{
6     "merchant_api_key": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
7     "intent": "CYBERSOURCE",
8     "user": "cus_62c5d7e8-7d80-43db-8385-c49872a23cb7",
9     "mode": "payment",
10    "currency": "PKR",
11    "amount": 10000,
12    "entry_mode": "raw",
13    "metadata": {
14      "source": "xxxx",
15      "order_id": "xxxx"
16    }
17  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_e454339b-b13f-45a2-ba44-60fba74c0c9b"	Tracker token. Pass this as the path parameter {{tracker_token}} in all subsequent steps.
data.tracker.client	String	M	"sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19"	Merchant client identifier.
data.tracker.environment	String	M	"sandbox"	Environment the tracker was created in.
data.tracker.state	String	M	"TRACKER_STARTED"	Initial tracker state.
data.tracker.payment_method_kind	String	M	"card"	Payment method type.
data.tracker.intent	String	M	"CYBERSOURCE"	Payment processor.
data.tracker.mode	String	M	"payment"	Transaction mode.
data.tracker.entry_mode	String	M	"raw"	Card entry mode.
data.tracker.customer	String	M	"cus_62c5d7e8-7d80-43db-8385-c49872a23cb7"	Customer token linked to this tracker.
data.tracker.next_actions.CYBERSOURCE.kind	String	M	"PAYER_AUTH_SETUP"	Next required action. Proceed to Step 3.

<code>data.tracker.purchase_totals.quote_amount.currency</code>	String	M	"PKR"	Quote currency.
<code>data.tracker.purchase_totals.quote_amount.amount</code>	Number	M	10000	Quote amount.
<code>data.tracker.purchase_totals.base_amount.currency</code>	String	M	"PKR"	Base currency.
<code>data.tracker.purchase_totals.base_amount.amount</code>	Number	M	10000	Base amount.
<code>data.tracker.purchase_totals.conversion_rate.rate</code>	Number	M	1	FX conversion rate applied.
<code>data.tracker.metadata.data.order_id</code>	String	O	"xxxx"	Echoed merchant order ID.
<code>data.tracker.metadata.data.source</code>	String	O	"xxxx"	Echoed transaction source.
<code>data.capabilities.CYBERSOURCE</code>	Boolean	M	true	Whether CyberSource is enabled for this merchant.
<code>status.errors</code>	Array	M	[]	List of errors. Empty on success.
<code>status.message</code>	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1  {
2    "data": {
3      "tracker": {
4        "token": "track_e454339b-b13f-45a2-ba44-60fba74c0c9b",
5        "client": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
6        "environment": "sandbox",
7        "state": "TRACKER_STARTED",
8        "payment_method_kind": "card",
9        "intent": "CYBERSOURCE",
10       "mode": "payment",
11       "entry_mode": "raw",
12       "customer": "cus_62c5d7e8-7d80-43db-8385-c49872a23cb7",
13       "next_actions": {
14         "CYBERSOURCE": { "kind": "PAYER_AUTH_SETUP" },
15         "MPGS": { "kind": "CREATE_SESSION" },
16         "PAYFAST": { "kind": "CUSTOMER_VALIDATE" }
17       },
18       "purchase_totals": {
19         "quote_amount": { "currency": "PKR", "amount": 10000 },
20         "base_amount": { "currency": "PKR", "amount": 10000 },
21         "conversion_rate": { "base_currency": "PKR", "quote_currency":
22         "PKR", "rate": 1 }
23       },
24       "metadata": { "data": { "order_id": "xxxx", "source": "xxxx" } }
25     },
26     "capabilities": { "CYBERSOURCE": true, "MPGS": true, "PAYFAST":
27     true }
28   },
29   "status": { "errors": [], "message": "success" }
30 }

```

Step 3. Payer Authentication Setup

Payer Authentication Setup

Description: Submits the raw card details against the tracker to initiate CyberSource Payer Authentication Setup. This call returns an `access_token` (the Cardinal Commerce JWT for device data collection) and a `device_data_collection_url`, both of which are fed into the Atoms module in Step 5. Also returns a `req_` request token needed by Atoms.

Critical: This endpoint is called against the **Simpaisa Secure Gateway** (`sandbox-securegateway.simpaisa.com`), **not** directly against `sandbox.api.getsafepay.com` . The path parameter `{{tracker_token}}` is the `data.tracker.token` returned in Step 2.

Method: POST

Endpoint: `/card/order/payments/v3/{{tracker_token}}`

Base URL (Sandbox): `<https://sandbox-securegateway.simpaisa.com >`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
{{tracker_token}}	Path	String	M	"track_e454339b-..."	Tracker token from Step 2.
payload.payment_method.card.card_number	Body	String	M	"5200000000001096"	Full card number (PAN). Transmitted via Simpaisa secure gateway only.
payload.payment_method.card.expiration_month	Body	String	M	"12"	Card expiration month (2-digit).
payload.payment_method.card.expiration_year	Body	String	M	"2028"	Card expiration year (4-digit).
payload.payment_method.card.cvv	Body	String	M	"123"	Card verification value.

Sample Request (cURL)

```
1 curl --request POST \  
2 --url "https://sandbox-securegateway.simpaisa.com/card/order/payments/v3/{{tracker_token}}" \  
3 --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4 --header "Content-Type: application/json" \  
5 --data '{  
6   "payload": {  
7     "payment_method": {  
8       "card": {  
9         "card_number": "5200000000001096",  
10        "expiration_month": "12",  
11        "expiration_year": "2028",  
12        "cvv": "123"  
13      }  
14    }  
15  }  
}
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
<code>data.tracker.token</code>	String	M	"track_e454339b-..."	Tracker token (unchanged).
<code>data.tracker.state</code>	String	M	"TRACKER_STARTED"	Tracker state (unchanged at this step).
<code>data.tracker.next_actions.CYBERSOURCE.kind</code>	String	M	"PAYER_AUTH_ENROLLMENT"	Next required action.
<code>data.tracker.next_actions.CYBERSOURCE.request_id</code>	String	M	"req_9c2b4b8d-..."	Request ID for the enrollment step.
<code>data.action.token</code>	String	M	"req_9c2b4b8d-2056-44be-9f33-5e311227de90"	Action request token.
<code>data.action.payment_method.token</code>	String	M	"card_9aed44ab-..."	Tokenized card reference (internal, not the <code>pm_token</code>).
<code>data.action.payment_method.expiration_month</code>	String	M	"12"	Card expiration month.
<code>data.action.payment_method.expiration_year</code>	String	M	"2028"	Card expiration year.
<code>data.action.payment_method.card_type_code</code>	String	M	"002"	Numeric card type code.
<code>data.action.payment_method.card_type</code>	String	M	"Mastercard"	Card network.
<code>data.action.payment_method.bin_number</code>	String	M	"520000"	First six digits of the card (BIN).
<code>data.action.payment_method.last_four</code>	String	M	"1096"	Last four digits of the card.
<code>data.action.payer_authentication_setup.access_token</code>	String	M	"eyJhbGciOi..."	Cardinal Commerce JWT for device data collection. Pass as <code>deviceDataCollectionJWT</code> in the Atoms module (Step 5).

<code>data.action.payer_authentication_setup.device_data_collection_url</code>	String	M	<code>"<https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect>"</code>	Device data collection URL. Pass as <code>deviceDataCollectionURL</code> in Atoms (Step 5). This URL differs between sandbox and production.
<code>status.errors</code>	Array	M	<code>[]</code>	List of errors. Empty on success.
<code>status.message</code>	String	M	<code>"success"</code>	Response status message.

Sample Response

Success (HTTP 200):

```
1 {
2   "data": {
3     "tracker": {
4       "token": "track_e454339b-b13f-45a2-ba44-60fba74c0c9b",
5       "client": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
6       "environment": "sandbox",
7       "state": "TRACKER_STARTED",
8       "payment_method_kind": "card",
9       "intent": "CYBERSOURCE",
10      "mode": "payment",
11      "entry_mode": "raw",
12      "customer": "cus_62c5d7e8-7d80-43db-8385-c49872a23cb7",
13      "next_actions": {
14        "CYBERSOURCE": {
15          "kind": "PAYER_AUTH_ENROLLMENT",
16          "request_id": "req_9c2b4b8d-2056-44be-9f33-5e311227de90"
17        }
18      },
19      "purchase_totals": {
20        "quote_amount": { "currency": "PKR", "amount": 10000 },
21        "base_amount": { "currency": "PKR", "amount": 10000 },
22        "conversion_rate": { "base_currency": "PKR", "quote_currency":
23        "PKR", "rate": 1 }
24      },
25      "metadata": { "data": { "order_id": "xxxx", "source": "xxxx" } }
26    },
27    "action": {
28      "token": "req_9c2b4b8d-2056-44be-9f33-5e311227de90",
29      "payment_method": {
30        "token": "card_9aed44ab-70b5-4385-9f8a-c21ef759c386",
31        "expiration_month": "12",
32        "expiration_year": "2028",
33        "card_type_code": "002",
34        "card_type": "Mastercard",
35        "bin_number": "520000",
36        "last_four": "1096"
37      },
38      "payer_authentication_setup": {
39        "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
40        "device_data_collection_url":
41        "https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect"
42      }
43    },
44    "status": { "errors": [], "message": "success" }
45  }
```

Step 4. Time-Based Token

Time-Based Token

Description: Generates a short-lived client authentication token used to authorize the Atoms SDK component on the frontend. This token is passed as `authToken` in the Atoms module (Step 5). Call this immediately before initializing Atoms as the token expires quickly.

Method: `POST`

Endpoint: `/client/passport/v1/token`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/client/passport/v1/token" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4   --header "Content-Type: application/json" \  
5   --data '{}'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data	String	M	"nTAvYaQ304ihCLAE3kLsJS7SMgy0C3gATn_Jz1wmy-Fa_GbPZ0mv2xS_2WQoQJmp0JfnEsDXdA=="	Time-based client auth token. Pass as authToken in the Atoms module (Step 5).

Sample Response

Success (HTTP 200):

```
1 {  
2   "data": "nTAvYaQ304ihCLAE3kLsJS7SMgy0C3gATn_Jz1wmy-Fa_GbPZ0mv2xS_2WQoQJmp0JfnEsDXdA=="  
3 }
```

Step 5. Atoms Module (Frontend SDK)

Atoms Module — PayerAuthentication Component

Description: Safepay Atoms is a frontend SDK (Web Component and React Component) that manages Device Data Collection and the 3DS Challenge flow. It must be initialized on the frontend with values collected from Steps 2, 3, and 4. The Atoms module handles card enrollment check, device fingerprinting, and cardholder challenge presentation entirely — no additional backend enrollment API is required.

In Flow 1, both card capture (`do_capture`) and card tokenization (`do_card_on_file`) are performed via this component.

`do_card_on_file` **must always be true** when tokenizing a card.

NPM Package: @sfpy/atoms — <https://www.npmjs.com/package/@sfpy/atoms>

Documentation: [Safepay Atoms Overview](#)

Component Configuration

```
1 <PayerAuthentication  
2   environment="sandbox"  
3   tracker="{{tracker_token}}"
```

```

4   authToken="{{time_based_token}}"
5   deviceDataCollectionJWT="{{access_token}}"
6   deviceDataCollectionURL="{{device_data_collection_url}}"
7   authorizationOptions={{
8     do_capture: true,      // true = perform sale (auth + capture in one step); false
= auth-only
9     do_card_on_file: true // MUST be true whenever tokenizing a card
10  }}
11  imperativeRef={authRef}
12  onPayerAuthenticationFailure={handleFailure}
13  onPayerAuthenticationSuccess={handleSuccess}
14  onPayerAuthenticationFrictionless={handleSuccess}
15  onPayerAuthenticationUnavailable={handleFailure}
16  onSafepayError={handleFailure}
17 />

```

Component Props

Prop	Type	M/O/C	Source	Description
<code>environment</code>	String	M	Config	"sandbox" or "production". Must match the environment of the tracker.
<code>tracker</code>	String	M	Step 2 response — <code>data.tracker.token</code>	The <code>track_</code> token from the Payment API.
<code>authToken</code>	String	M	Step 4 response — <code>data</code>	The time-based client auth token.
<code>deviceDataCollectionJWT</code>	String	M	Step 3 response — <code>data.action.payer_authentication_setup.access_token</code>	Cardinal Commerce JWT for device data collection.
<code>deviceDataCollectionURL</code>	String	M	Step 3 response — <code>data.action.payer_authentication_setup.device_data_collection_url</code>	Device data collection endpoint. This URL is different for sandbox vs production.
<code>authorizationOptions.do_capture</code>	Boolean	M	Implementation decision	<code>true</code> to perform auth + capture in one step. <code>false</code> for auth-only (proceed to Step 6 Capture).
<code>authorizationOptions.do_card_on_file</code>	Boolean	M	Always <code>true</code> in tokenization flows	Must be <code>true</code> whenever the card should be tokenized and saved to the customer's wallet.
<code>imperativeRef</code>	Ref	M	Local ref	React ref used to imperatively trigger authentication.
<code>onPayerAuthenticationSuccess</code>	Function	M	Implementation	Callback on successful 3DS authentication.
<code>onPayerAuthenticationFrictionless</code>	Function	M	Implementation	Callback when card passes frictionlessly (no challenge).

<code>onPayerAuthenticationFailure</code>	Function	M	Implementation	Callback on 3DS authentication failure.
<code>onPayerAuthenticationUnavailable</code>	Function	M	Implementation	Callback when 3DS is unavailable for the card.
<code>onSafepayError</code>	Function	M	Implementation	Callback on SDK-level error.

Note: If `do_capture: true`, the payment is finalized within Atoms and Step 6 (Capture) can be skipped. If `do_capture: false`, the transaction is auth-only and Step 6 must be called to move funds.

Step 6. Capture (Optional)

Capture

Description: Finalizes an auth-only transaction by instructing the bank to move the reserved funds from the customer's card to the merchant. Only call this step if `do_capture` was set to `false` in the Atoms module (Step 5). If `do_capture` was `true`, the capture was already completed via Atoms and this step must be skipped.

The `{{tracker_token}}` path parameter is the `data.tracker.token` returned in Step 2.

Method: POST

Endpoint: `/order/payments/v3/{{tracker_token}}`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
<code>X-SFPY-MERCHANT-SECRET</code>	Header	String	M	<code>{{merchant_secret_key}}</code>	Merchant authentication secret key.
<code>Content-Type</code>	Header	String	M	<code>application/json</code>	Request payload format.
<code>{{tracker_token}}</code>	Path	String	M	<code>"track_f8334909-..."</code>	Tracker token from Step 2.

Sample Request (cURL)

```
1 curl --request POST \
2   --url "{{baseUrl}}/order/payments/v3/{{tracker_token}}" \
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{}'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
-----------	-----------	-------	--------------	-------------

<code>data.tracker.token</code>	String	M	"track_f8334909-6e41-4e34-8f8c-3a083cde032f"	Tracker token.
<code>data.tracker.state</code>	String	M	"TRACKER_ENDED"	Final state confirming payment capture completed.
<code>data.tracker.entry_mode</code>	String	M	"raw"	Card entry mode.
<code>data.tracker.next_actions.CYBERSOURCE.kind</code>	String	M	"N00P"	No further actions required.
<code>data.tracker.next_actions.CYBERSOURCE.request_id</code>	String	M	"req_7a24924a-..."	Request ID of last action.
<code>data.tracker.purchase_totals.quote_amount.amount</code>	Number	M	50	Captured amount.
<code>data.tracker.purchase_totals.quote_amount.currency</code>	String	M	"PKR"	Currency.
<code>data.tracker.authorization.token</code>	String	M	"auth_112c0ab4-f5ad-481a-bfeb-def6f161704f"	Authorization token for this transaction.
<code>data.tracker.authorization.tracker</code>	String	M	"track_f8334909-..."	Tracker linked to authorization.
<code>data.tracker.authorization.attempt</code>	String	M	"attemp_b7da9d73-..."	Attempt ID.
<code>data.tracker.authorization.intent</code>	String	M	"CYBERSOURCE"	Processor used.
<code>data.tracker.authorization.cybersource_rid</code>	String	M	"7636467483736904704807"	CyberSource reference ID.
<code>data.tracker.authorization.is_reversible</code>	Boolean	M	true	Whether the transaction can still be reversed.
<code>data.tracker.authorization.totals.amount</code>	Number	M	50	Authorized amount.
<code>data.tracker.authorization.totals.currency</code>	String	M	"PKR"	Currency of authorized amount.
<code>data.tracker.authorization.created_at.seconds</code>	Number	M	1763646750	Unix timestamp of authorization creation.

<code>data.tracker.authorization.updated_at.seconds</code>	Number	M	1763646750	Unix timestamp of last authorization update.
<code>data.action.token</code>	String	M	"req_7a24924a-..."	Action token for this step.
<code>data.action.payment_method.token</code>	String	M	"card_d3756b2c-..."	Card token used in the payment.
<code>data.action.payment_method.card_type</code>	String	M	"Mastercard"	Card network.
<code>data.action.payment_method.last_four</code>	String	M	"1096"	Last four digits of card.
<code>status.errors</code>	Array	M	[]	Empty on success.
<code>status.message</code>	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1  {
2    "data": {
3      "tracker": {
4        "token": "track_f8334909-6e41-4e34-8f8c-3a083cde032f",
5        "client": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
6        "environment": "sandbox",
7        "state": "TRACKER_ENDED",
8        "payment_method_kind": "card",
9        "intent": "CYBERSOURCE",
10       "mode": "payment",
11       "entry_mode": "raw",
12       "customer": "cus_4a14f3ba-4563-40c9-aa51-a66a16e929b1",
13       "next_actions": {
14         "CYBERSOURCE": { "kind": "N00P", "request_id": "req_7a24924a-
15b3-42d4-976c-c84e8cc57a93" }
16       },
17       "purchase_totals": {
18         "quote_amount": { "currency": "PKR", "amount": 50 },
19         "base_amount": { "currency": "PKR", "amount": 50 },
20         "conversion_rate": { "base_currency": "PKR", "quote_currency":
21 "PKR", "rate": 1 }
22       },
23       "metadata": { "data": { "order_id": "41231X", "source": "yango"
24 } },
25     },
26     "authorization": {
27       "token": "auth_112c0ab4-f5ad-481a-bfeb-def6f161704f",
28       "tracker": "track_f8334909-6e41-4e34-8f8c-3a083cde032f",
29       "attempt": "attemp_b7da9d73-0cd5-4944-8b26-aa9282123bc9",
30       "intent": "CYBERSOURCE",
31       "cybersource_rid": "7636467483736904704807",
32       "is_reversible": true,
33       "totals": { "currency": "PKR", "amount": 50 },
34       "created_at": { "seconds": 1763646750 },
35       "updated_at": { "seconds": 1763646750 }
36     },
37     "action": {
38       "token": "req_7a24924a-15b3-42d4-976c-c84e8cc57a93",
39       "payment_method": {
40         "token": "card_d3756b2c-b4bb-4abc-affc-dfacf58da121",
41         "expiration_month": "12",
42         "expiration_year": "2028",
43         "card_type_code": "002",
44         "card_type": "Mastercard",
45         "bin_number": "520000",
46         "last_four": "1096"
47       }
48     },
49     "status": { "errors": [], "message": "success" }
50   }

```


Description: Fetches all payment method tokens (`pm_` tokens) saved against a customer. This is a mandatory call after completing the Atoms flow — it is the only way to retrieve the `pm_` token that is required to initiate token-based charges in Flow 2 and Flow 3. The `{{customer_token}}` path parameter is the `cus_` token from Step 1.

NOTE: The pm token is in the parameter of the response of the API which is the `wallet.token` param and the TMS(Cybersource Token) is in the param of `cybersource.token`

Method: GET

Endpoint: `/user/customers/v1/{{customer_token}}/wallet/`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
<code>X-SFPY-MERCHANT-SECRET</code>	Header	String	M	<code>{{merchant_secret_key}}</code>	Merchant authentication secret key.
<code>{{customer_token}}</code>	Path	String	M	<code>"cus_2fc20245-..."</code>	Customer token from Step 1.
<code>limit</code>	Query	Number	M	<code>20</code>	Maximum number of results to return.
<code>page</code>	Query	Number	M	<code>1</code>	Page offset for pagination.

Sample Request (cURL)

```
1 curl --request GET \  
2   --url "{{baseUrl}}/user/customers/v1/{{customer_token}}/wallet/?  
3   limit=20&page=1" \  
   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}"
```

Response Parameters

Response Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Data Type	M/O/C	Sample Value	Description
<code>data.wallet</code>	Array	M	<code>[...]</code>	Array of saved payment method objects for this customer.
<code>data.wallet[].token</code>	String	M	<code>"pm_5543b042-bd9c-4853-995b-2405b7c72dd9"</code>	The <code>pm_</code> token. This is the value passed as <code>tokenized_card.token</code> in Flow 2 and Flow 3.
<code>data.wallet[].customer</code>	String	M	<code>"cus_a03d1304-d088-4a60-801f-b6a3d44b445d"</code>	Customer token this payment method belongs to.
<code>data.wallet[].merchant_api_key</code>	String	M	<code>"sec_efbce898-4cc4-4c00-ba45-</code>	Merchant the payment method is registered

			f00001804a60"	under.
data.wallet[].kind	Number	M	1	Payment method type code.
data.wallet[].max_usage	Number	M	1	Maximum number of times this payment method can be used.
data.wallet[].usage_interval	Number	M	0	Usage interval in seconds. 0 means no restriction.
data.wallet[].usage_count	Number	M	0	Number of times this payment method has been used to date.
data.wallet[].is_deleted	Boolean	M	false	Whether this payment method has been soft-deleted.
data.wallet[].expires_at.seconds	Number	M	1700810946	Unix timestamp after which this payment method expires.
data.wallet[].created_at.seconds	Number	M	1700724546	Unix timestamp when this payment method was created.
data.wallet[].updated_at.seconds	Number	M	1700724546	Unix timestamp of last update to this payment method.
data.wallet[].billing_address.token	String	M	"address_165864ee-c9b4-4e6f-97a1-c6bf56b6bce8"	Unique token for the billing address.
data.wallet[].billing_address.owner	String	M	"cus_a03d1304-d088-4a60-801f-b6a3d44b445d"	Customer token who owns this billing address.
data.wallet[].billing_address.street1	String	O	"house"	Street address line 1.
data.wallet[].billing_address.city	String	O	"qhouse"	City.
data.wallet[].billing_address.state	String	O	"TX"	State or province.
data.wallet[].billing_address.postal_code	String	O	"77027"	Postal or ZIP code.
data.wallet[].billing_address.country	String	M	"PK"	ISO 3166-1 alpha-2 country code.
data.wallet[].billing_address.is_default	Boolean	M	false	Whether this is the customer's default billing address.
data.wallet[].billing_address.created_at.seconds	Number	M	1700724546	Unix timestamp when billing address was

at.seconds				created.
data.wallet[].billing_address.updated_at.seconds	Number	M	1700724546	Unix timestamp of last billing address update.
data.wallet[].cybersource.token	String	M	"tms_8ca63428-5d80-4542-8a93-401d797b91ff"	Internal CyberSource TMS token for this card.
data.wallet[].cybersource.customer_payment_method	String	M	"pm_5543b042-bd9c-4853-995b-2405b7c72dd9"	Reference back to the parent pm_ token.
data.wallet[].cybersource.fingerprint	String	M	"17eec1263ea9b3a244be72b5ec27bfaf02d84e180085ae67da0b981c79231eb5"	Unique card fingerprint hash. Can be used to detect duplicate cards across customers.
data.wallet[].cybersource.scheme	Number	M	1	Card scheme code. 4 = Visa, 5 = Mastercard. This is main identifier that we need to distinguish whether the card is visa or what
data.wallet[].cybersource.bin	String	M	"411111"	First six digits of the card (Bank Identification Number).
data.wallet[].cybersource.last_four	String	M	"1111"	Last four digits of the card.
data.wallet[].cybersource.expiry_month	String	M	"12"	Card expiry month (2-digit).
data.wallet[].cybersource.expiry_year	String	M	"2029"	Card expiry year (4-digit).
data.wallet[].cybersource.created_at.seconds	Number	M	1700724546	Unix timestamp when the CyberSource token was created.
data.wallet[].cybersource.updated_at.seconds	Number	M	1700724546	Unix timestamp of last CyberSource token update.
data.count	Number	M	2	Total number of payment methods returned.
status.errors	Array	M	[]	List of errors. Empty on success.
status.message	String	M	"success"	

Sample Response

Success (HTTP 200):

```

1 {
2   "data": {
3     "wallet": [
4       {
5         "token": "pm_5543b042-bd9c-4853-995b-2405b7c72dd9",

```

```

6         "customer": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
7         "merchant_api_key": "sec_efbce898-4cc4-4c00-ba45-
f00001804a60",
8         "kind": 1,
9         "max_usage": 1,
10        "usage_interval": 0,
11        "is_deleted": false,
12        "billing_address": {
13            "token": "address_165864ee-c9b4-4e6f-97a1-c6bf56b6bce8",
14            "owner": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
15            "street1": "house",
16            "city": "qhouse",
17            "country": "PK",
18            "is_default": false,
19            "created_at": {
20                "seconds": 1700724546
21            },
22            "updated_at": {
23                "seconds": 1700724546
24            }
25        },
26        "cybersource": {
27            "token": "tms_8ca63428-5d80-4542-8a93-401d797b91ff",
28            "customer_payment_method": "pm_5543b042-bd9c-4853-995b-
2405b7c72dd9",
29            "fingerprint":
"17ec1263ea9b3a244be72b5ec27bfaf02d84e180085ae67da0b981c79231eb5",
30            "scheme": 1,
31            "bin": "411111",
32            "last_four": "1111",
33            "expiry_month": "12",
34            "expiry_year": "2029",
35            "created_at": {
36                "seconds": 1700724546
37            },
38            "updated_at": {
39                "seconds": 1700724546
40            }
41        },
42        "usage_count": 0,
43        "expires_at": {
44            "seconds": 1700810946
45        },
46        "created_at": {
47            "seconds": 1700724546
48        },
49        "updated_at": {
50            "seconds": 1700724546
51        }
52    },
53    {
54        "token": "pm_3b4a7c47-99cc-4fda-9b4c-b46b11e2cd18",
55        "customer": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
56        "merchant_api_key": "sec_efbce898-4cc4-4c00-ba45-
f00001804a60",
57        "kind": 1,
58        "max_usage": 1,
59        "usage_interval": 0,
60        "is_deleted": false,
61        "billing_address": {
62            "token": "address_25f9c917-9d73-4e45-964a-916df314d388",
63            "owner": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
64            "street1": "house",
65            "city": "Houston",
66            "state": "TX",
67            "postal_code": "77027",
68            "country": "US",
69            "is_default": false,
70            "created_at": {
71                "seconds": 1700724454
72            },
73            "updated_at": {
74                "seconds": 1700724454
75            }
76        },
77        "cybersource": {
78            "token": "tms_eb5660d2-e0d6-4898-8fb0-51f53ff49022",
79            "customer_payment_method": "pm_3b4a7c47-99cc-4fda-9b4c-
b46b11e2cd18",
80            "fingerprint":
"7d7c3ca2d742e36733aca5a7a595ed822eaf2b7295cc9599819e3928ce85f9c",
81            "scheme": 2,
82            "bin": "520000",
83            "last_four": "1096",
84            "expiry_month": "12",
85            "expiry_year": "2028",
86            "created_at": {
87                "seconds": 1700724455
88            },
89            "updated_at": {
90                "seconds": 1700724455
91            }
92        },
93        "usage_count": 0,
94        "expires_at": {

```

```
95         "seconds": 1700810855
96     },
97     "created_at": {
98         "seconds": 1700724455
99     },
100    "updated_at": {
101        "seconds": 1700724455
102    }
103 }
104 ],
105 "count": 2
106 },
107 "status": {
108     "errors": [],
109     "message": "success"
110 }
111 }
```

Step 8. Find Payment Method (Optional)

Find Payment Method

Description: Retrieves the full details of a single saved payment method by combining the customer token and the `pm_` token.

Method: GET

Endpoint: `/user/customers/v1/{{customer_token}}/wallet/{{pm_token}}`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
{{customer_token}}	Path	String	M	"cus_2fc20245-..."	Customer token from Step 1.
{{pm_token}}	Path	String	M	"pm_bea00774-..."	Payment method token from Step 7.

Sample Request (cURL)

```
1 curl --request GET \
2   --url "
3   {{baseUrl}}/user/customers/v1/{{customer_token}}/wallet/{{pm_token}}" \
   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}"
```

Response Parameters

Response Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Data Type	M/O/C	Sample Value	Description
-----------	-----------	-------	--------------	-------------

<code>data.token</code>	String	M	"pm_3b4a7c47-99cc-4fda-9b4c-b46b11e2cd18"	The <code>pm_</code> token for this payment method. This is the value passed as <code>tokenized_card.token</code> in Flow 2 and Flow 3.
<code>data.customer</code>	String	M	"cus_a03d1304-d088-4a60-801f-b6a3d44b445d"	Customer token this payment method belongs to.
<code>data.merchant_api_key</code>	String	M	"sec_efbce898-4cc4-4c00-ba45-f00001804a60"	Merchant the payment method is registered under.
<code>data.kind</code>	Number	M	1	Payment method type code.
<code>data.max_usage</code>	Number	M	1	Maximum number of times this payment method can be used.
<code>data.usage_interval</code>	Number	M	0	Usage interval in seconds. 0 means no restriction.
<code>data.usage_count</code>	Number	M	0	Number of times this payment method has been used to date.
<code>data.is_deleted</code>	Boolean	M	false	Whether this payment method has been soft-deleted.
<code>data.expires_at.seconds</code>	Number	M	1700810855	Unix timestamp after which this payment method expires.
<code>data.created_at.seconds</code>	Number	M	1700724455	Unix timestamp when this payment method was created.
<code>data.updated_at.seconds</code>	Number	M	1700724455	Unix timestamp of last update to this payment method.
<code>data.billing_addresses.token</code>	String	M	"address_25f9c917-9d73-4e45-964a-916df314d388"	Unique token for the billing address.
<code>data.billing_addresses.owner</code>	String	M	"cus_a03d1304-d088-4a60-801f-b6a3d44b445d"	Customer token who owns this billing address.
<code>data.billing_addresses.street1</code>	String	O	"house"	Street address line 1.
<code>data.billing_addresses.city</code>	String	O	"Houston"	City.
<code>data.billing_addresses.state</code>	String	O	"TX"	State or province.

<code>data.billing_address.postal_code</code>	String	O	"77027"	Postal or ZIP code.
<code>data.billing_address.country</code>	String	M	"US"	ISO 3166-1 alpha-2 country code.
<code>data.billing_address.is_default</code>	Boolean	M	false	Whether this is the customer's default billing address.
<code>data.billing_address.created_at.seconds</code>	Number	M	1700724454	Unix timestamp when billing address was created.
<code>data.billing_address.updated_at.seconds</code>	Number	M	1700724454	Unix timestamp of last billing address update.
<code>data.cybersource.token</code>	String	M	"tms_eb5660d2-e0d6-4898-8fb0-51f53ff49022"	Internal CyberSource TMS token for this card.
<code>data.cybersource.customer_payment_method</code>	String	M	"pm_3b4a7c47-99cc-4fda-9b4c-b46b11e2cd18"	Reference back to the parent pm_ token.
<code>data.cybersource.fingerprint</code>	String	M	"7d7c3ca2d742e36733aca5a7a595ed822eaaf2b7295cc9599819e3928ce85fc9"	Unique card fingerprint hash. Can be used to detect duplicate cards across customers.
<code>data.cybersource.scheme</code>	Number	M	2	Card scheme code. 4 = Visa, 5 = Mastercard. This is main identifier that we need to distinguish whether the card is visa or what
<code>data.cybersource.bin</code>	String	M	"520000"	First six digits of the card (Bank Identification Number).
<code>data.cybersource.last_four</code>	String	M	"1096"	Last four digits of the card.
<code>data.cybersource.expiry_month</code>	String	M	"12"	Card expiry month (2-digit).
<code>data.cybersource.expiry_year</code>	String	M	"2028"	Card expiry year (4-digit).
<code>data.cybersource.created_at.seconds</code>	Number	M	1700724455	Unix timestamp when the CyberSource token was created.
<code>data.cybersource.updated_at.seconds</code>	Number	M	1700724455	Unix timestamp of last CyberSource token update.
<code>status.errors</code>	Array	M	[]	List of errors. Empty on success.

status.message	String	M	"success"	
----------------	--------	---	-----------	--

Sample Response

Success (HTTP 200):

```
1 {
2   "data": {
3     "token": "pm_3b4a7c47-99cc-4fda-9b4c-b46b11e2cd18",
4     "customer": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
5     "merchant_api_key": "sec_efbce898-4cc4-4c00-ba45-f00001804a60",
6     "kind": 1,
7     "max_usage": 1,
8     "usage_interval": 0,
9     "is_deleted": false,
10    "billing_address": {
11      "token": "address_25f9c917-9d73-4e45-964a-916df314d388",
12      "owner": "cus_a03d1304-d088-4a60-801f-b6a3d44b445d",
13      "street1": "house",
14      "city": "Houston",
15      "state": "TX",
16      "postal_code": "77027",
17      "country": "US",
18      "is_default": false,
19      "created_at": {
20        "seconds": 1700724454
21      },
22      "updated_at": {
23        "seconds": 1700724454
24      }
25    },
26    "cybersource": {
27      "token": "tms_eb5660d2-e0d6-4898-8fb0-51f53ff49022",
28      "customer_payment_method": "pm_3b4a7c47-99cc-4fda-9b4c-
29      b46b11e2cd18",
30      "fingerprint":
31      "7d7c3ca2d742e36733aca5a7a595ed822eaf2b7295cc9599819e3928ce85fc9",
32      "scheme": 2,
33      "bin": "520000",
34      "last_four": "1096",
35      "expiry_month": "12",
36      "expiry_year": "2028",
37      "created_at": {
38        "seconds": 1700724455
39      },
40      "updated_at": {
41        "seconds": 1700724455
42      }
43    },
44    "usage_count": 0,
45    "expires_at": {
46      "seconds": 1700810855
47    },
48    "created_at": {
49      "seconds": 1700724455
50    },
51    "updated_at": {
52      "seconds": 1700724455
53    }
54  },
55  "status": {
56    "errors": [],
57    "message": "success"
58  }
59 }
```

Step 9. Delete Payment Method

Delete Payment Method

Description: Soft-deletes a saved payment method from the customer's wallet. The `pm_` token is marked as deleted and can no longer be used to initiate charges. Returns the `pm_` token and a `deleted` flag confirming success.

Method: DELETE

Endpoint: `/user/customers/v1/{customer_token}/wallet/{pm_token}`

Authentication: Header — X-SFPY-MERCHANT-SECRET: `{{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
{{customer_token}}	Path	String	M	"cus_2fc20245-..."	Customer token.
{{pm_token}}	Path	String	M	"pm_bea00774-..."	Payment method token to delete.

Sample Request (cURL)

```
1 curl --request DELETE \  
2   --url " {{baseUrl}}/user/customers/v1/{{customer_token}}/wallet/{{pm_token}}" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}"
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.token	String	M	"pm_bea00774-22ba-44ce-82c5-3cdd1a342e89"	The pm_token that was deleted.
data.deleted	Boolean	M	true	true confirms the deletion was successful.
status.errors	Array	M	[]	Empty on success.
status.message	String	M	"success"	Response status message.

Sample Response

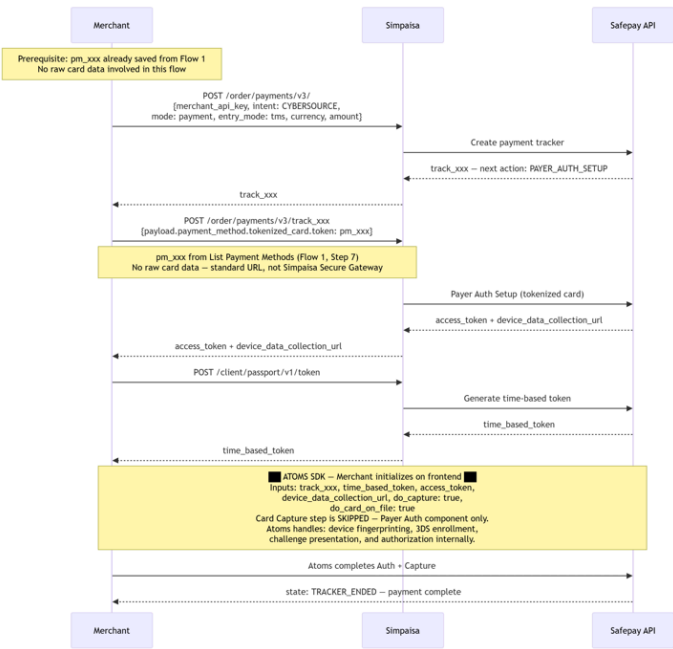
Success (HTTP 200):

```
1 {  
2   "data": {  
3     "token": "pm_bea00774-22ba-44ce-82c5-3cdd1a342e89",  
4     "deleted": true  
5   },  
6   "status": { "errors": [], "message": "success" }  
7 }
```

5. Flow 2 — Token Initiated Payment (With 3DS)

This flow charges a customer using a previously saved `pm_` token. The `entry_mode` is `"tms"` (tokenized). 3DS authentication is performed via the Atoms module. The card capture step in Atoms is skipped — only the Payer Authentication component is used. Obtain the `pm_` token by calling List Payment Methods (Flow 1, Step 7) before initiating this flow.

Sequence Diagram



Step 1. Create Payment Tracker (Payment API)

Payment API

Description: Creates a new tracker for a token-initiated payment. Set `entry_mode`: `"tms"` to indicate a saved token will be used. Note that `user` is **not** required in the request body for this flow — the customer association is resolved when the `pm_` token is submitted in Step 2.

Method: POST

Endpoint: `/order/payments/v3/`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.

merchant_api_key	Body	String	M	"sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1"	Merchant API key.
intent	Body	String	M	"CYBERSOURCE"	Payment processor.
mode	Body	String	M	"payment"	Transaction mode.
entry_mode	Body	String	M	"tms"	Set to "tms" for token-initiated payment.
currency	Body	String	M	"PKR"	ISO 4217 currency code.
amount	Body	Number	M	600000	Transaction amount in smallest currency unit.

Sample Request (cURL)

```

1 curl --request POST \
2   --url "{{baseUrl}}/order/payments/v3/" \
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{
6     "merchant_api_key": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
7     "intent": "CYBERSOURCE",
8     "mode": "payment",
9     "entry_mode": "tms",
10    "currency": "PKR",
11    "amount": 600000
12  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_61414d0f-7740-4297-903a-3aca4b04bca8"	Tracker token. Pass as <code>{{tracker_token}}</code> in Step 2.
data.tracker.state	String	M	"TRACKER_STARTED"	Initial tracker state.
data.tracker.entry_mode	String	M	"tms"	Confirms tokenized mode.
data.tracker.next_actions.CYBERSOURCE.kind	String	M	"PAYER_AUTH_SETUP"	Next required action — proceed to Step 2.

status.errors	Array	M	[]	Empty on success.
status.message	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```
1 {
2   "data": {
3     "tracker": {
4       "token": "track_61414d0f-7740-4297-903a-3aca4b04bca8",
5       "client": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
6       "environment": "sandbox",
7       "state": "TRACKER_STARTED",
8       "intent": "CYBERSOURCE",
9       "mode": "payment",
10      "entry_mode": "tms",
11      "next_actions": {
12        "CYBERSOURCE": { "kind": "PAYER_AUTH_SETUP" },
13        "MPGS": { "kind": "NOOP" }
14      },
15      "purchase_totals": {
16        "quote_amount": { "currency": "PKR", "amount": 600000 },
17        "base_amount": { "currency": "PKR", "amount": 600000 },
18        "conversion_rate": { "base_currency": "PKR", "quote_currency":
19        "PKR", "rate": 1 }
20      },
21      "metadata": {}
22    },
23    "capabilities": { "CYBERSOURCE": true, "MPGS": true }
24  },
25  "status": { "errors": [], "message": "success" }
26 }
```

Step 2. Payer Authentication Setup

Payer Authentication Setup

Description: Submits the saved `pm_` token to the tracker to initiate Payer Authentication Setup. The `tokenized_card.token` value is the `pm_` token retrieved from List Payment Methods (Flow 1, Step 7). This call is made against the standard Safepay base URL (not the Simpaisa secure gateway) because no raw card data is being transmitted.

Returns `access_token` and `device_data_collection_url` required for Atoms in Step 3.

Method: POST

Endpoint: `/order/payments/v3/{{tracker_token}}`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
{{tracker_token}}	Path	String	M	"track_61414d0f-..."	Tracker token from Step 1.

payload.payment_method.tokenized_card.token	Body	String	M	"pm_ad4b0519-c700-4c7f-8ed1-e0f06944bcab"	The pm_token from List Payment Methods (Flow 1, Step 7).
---	------	--------	---	---	--

Sample Request (cURL)

```
1 curl --request POST \
2   --url "{{baseUrl}}/order/payments/v3/{{tracker_token}}" \
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{
6     "payload": {
7       "payment_method": {
8         "tokenized_card": {
9           "token": "pm_ad4b0519-c700-4c7f-8ed1-e0f06944bcab"
10        }
11      }
12    }
13  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_61414d0f-..."	Tracker token.
data.tracker.customer	String	M	"user_d75110e4-..."	Customer resolved from the pm_token.
data.tracker.next_actions.CYBERSOURCE.kind	String	M	"PAYER_AUTH_ENROLLMENT"	Next required action.
data.tracker.next_actions.CYBERSOURCE.request_id	String	M	"req_ac4307d3-..."	Request ID for enrollment.
data.action.token	String	M	"req_ac4307d3-be62-4ea7-ab5f-e950125940df"	Action token.
data.action.payment_method.token	String	M	"card_f116867e-..."	Internal card token.
data.action.payment_method.pointer	String	M	"inst_ad4b0519-..."	Internal instrument pointer.
data.action.payment_method.card_type	String	M	"Visa"	Card network.
data.action.payment_method.last_four	String	M	"1111"	Last four digits.
data.action.payer_authentication_setup.access_token	String	M	"eyJhbGci..."	Pass as deviceDataCollectionJWT in Atoms (Step 3).
data.action.payer_authentication_setup.device_data_collection_url	String	M	"<https://centinelapistag.cardinalcommerce..."	Pass as deviceDataCollectionURL in Atoms (Step 3).

			rce.com/V1/Cruise/C ollect>"	Differs between sandbox and production.
status.errors	Array	M	[]	Empty on success.
status.message	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1  {
2    "data": {
3      "tracker": {
4        "token": "track_61414d0f-7740-4297-903a-3aca4b04bca8",
5        "client": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
6        "environment": "sandbox",
7        "state": "TRACKER_STARTED",
8        "intent": "CYBERSOURCE",
9        "mode": "payment",
10       "entry_mode": "tms",
11       "customer": "user_d75110e4-f52e-464d-96bd-490f75e7fc71",
12       "next_actions": {
13         "CYBERSOURCE": { "kind": "PAYER_AUTH_ENROLLMENT",
14           "request_id": "req_ac4307d3-be62-4ea7-ab5f-e950125940df" },
15         "MPGS": { "kind": "NOOP" }
16       },
17       "purchase_totals": {
18         "quote_amount": { "currency": "PKR", "amount": 600000 },
19         "base_amount": { "currency": "PKR", "amount": 600000 },
20         "conversion_rate": { "base_currency": "PKR", "quote_currency":
21           "PKR", "rate": 1 }
22       },
23       "metadata": {}
24     },
25     "action": {
26       "token": "req_ac4307d3-be62-4ea7-ab5f-e950125940df",
27       "payment_method": {
28         "token": "card_f116867e-955a-4d03-802d-1f4d06fe1921",
29         "pointer": "inst_ad4b0519-c700-4c7f-8ed1-e0f06944bcab",
30         "expiration_month": "12",
31         "expiration_year": "2025",
32         "card_type_code": "001",
33         "card_type": "Visa",
34         "bin_number": "411111",
35         "last_four": "1111"
36       },
37       "payer_authentication_setup": {
38         "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
39         "device_data_collection_url":
40           "https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect"
41       }
42     },
43     "status": { "errors": [], "message": "success" }
44   }

```

Step 3. Time-Based Token

Identical to Flow 1, Step 4. Call `POST /client/passport/v1/token` with the merchant secret key. The returned `data` string is passed as `authToken` in Atoms.

```

1  curl --request POST \
2    --url "{{baseUrl}}/client/passport/v1/token" \
3    --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4    --header "Content-Type: application/json" \
5    --data '{}'

```

Step 4. Atoms Module (Payer Auth Module Only — Card Capture Module Skipped)

Atoms Module — Payer Authentication Only

Description: In Flow 2, the Atoms module is initialized identically to Flow 1 with one critical difference: the card capture step is already completed internally during Payer Auth for `tms` mode. The customer's card data is not re-submitted — only the Payer Authentication component runs. Populate all props using values from Steps 1, 2, and 3 of this flow.

```

1  <PayerAuthentication
2    environment="sandbox"
3    tracker="{{tracker_token}}"
4    authToken="{{time_based_token}}"
5    deviceDataCollectionJWT="{{access_token}}"

```

```

6   deviceDataCollectionURL="{{device_data_collection_url}}"
7   authorizationOptions={{
8     do_capture: true,
9     do_card_on_file: true // Must remain true for tokenized flows
10  }}
11  imperativeRef={authRef}
12  onPayerAuthenticationFailure={handleFailure}
13  onPayerAuthenticationSuccess={handleSuccess}
14  onPayerAuthenticationFrictionless={handleSuccess}
15  onPayerAuthenticationUnavailable={handleFailure}
16  onSafepayError={handleFailure}
17 />

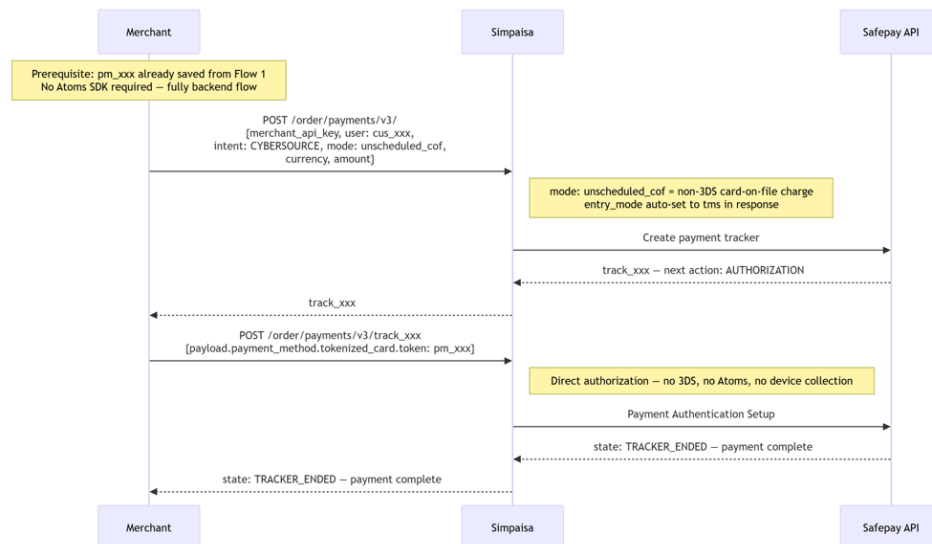
```

Prop	Source in This Flow
tracker	Step 1 response — data.tracker.token
authToken	Step 3 response — data
deviceDataCollectionJWT	Step 2 response — data.action.payer_authentication_setup.access_token
deviceDataCollectionURL	Step 2 response — data.action.payer_authentication_setup.device_data_collection_url

6. Flow 3 — Token Initiated Payment (Without 3DS)

This flow charges a customer using a saved `pm_` token without a 3DS challenge. The `mode` is `"unscheduled_cof"` (unscheduled card-on-file). The `user` parameter must be the customer token that owns the `pm_` token. No Atoms module is needed — the payment completes after the Payment Authentication Setup call.

Sequence Diagram



Step 1. Create Payment Tracker (Payment API)

Payment API

Description: Creates a tracker for an unscheduled card-on-file payment. Set `mode: "unscheduled_cof"` and include the `user` field with the customer token that owns the target `pm_` token. The tracker response will show `next_actions.CYBERSOURCE.kind:`

"AUTHORIZATION" , indicating the system will attempt direct authorization without 3DS.

Method: POST

Endpoint: /order/payments/v3/

Authentication: Header — X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
merchant_api_key	Body	String	M	"sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1"	Merchant API key.
user	Body	String	M	"cus_b4afd638-41bf-4831-af07-587b51269f38"	Customer token that owns the pm_ token to be charged.
intent	Body	String	M	"CYBERSOURCE"	Payment processor.
mode	Body	String	M	"unscheduled_cof"	Set to "unscheduled_cof" for non-3DS token payment.
currency	Body	String	M	"PKR"	ISO 4217 currency code.
amount	Body	Number	M	10050	Transaction amount in smallest currency unit.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/order/payments/v3/" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4   --header "Content-Type: application/json" \  
5   --data '{  
6     "merchant_api_key": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",  
7     "user": "cus_b4afd638-41bf-4831-af07-587b51269f38",  
8     "intent": "CYBERSOURCE",  
9     "mode": "unscheduled_cof",  
10    "currency": "PKR",  
11    "amount": 10050  
12  }'
```


Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_da499080-3c67-4f82-8f33-c4338c8020c5"	Tracker token. Pass as <code>{{tracker_token}}</code> in Step 2.
data.tracker.mode	String	M	"unscheduled_cof"	Confirms unscheduled COF mode.
data.tracker.entry_mode	String	M	"tms"	Automatically set to <code>tms</code> for COF flows.
data.tracker.customer	String	M	"cus_b4afd638-..."	Customer token confirmed.
data.tracker.next_actions.CYBERSOURCE.kind	String	M	"AUTHORIZATION"	Next action is direct authorization. Proceed to Step 2.
status.errors	Array	M	[]	Empty on success.
status.message	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```
1 {
2   "data": {
3     "tracker": {
4       "token": "track_da499080-3c67-4f82-8f33-c4338c8020c5",
5       "client": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
6       "environment": "sandbox",
7       "state": "TRACKER_STARTED",
8       "intent": "CYBERSOURCE",
9       "mode": "unscheduled_cof",
10      "entry_mode": "tms",
11      "customer": "cus_b4afd638-41bf-4831-af07-587b51269f38",
12      "next_actions": {
13        "CYBERSOURCE": { "kind": "AUTHORIZATION" },
14        "MPGS": { "kind": "NOOP" }
15      },
16      "purchase_totals": {
17        "quote_amount": { "currency": "PKR", "amount": 10050 },
18        "base_amount": { "currency": "PKR", "amount": 10050 },
19        "conversion_rate": { "base_currency": "PKR", "quote_currency":
20        "PKR", "rate": 1 }
21      },
22      "metadata": {}
23    },
24    "capabilities": { "CYBERSOURCE": true, "MPGS": true }
25  },
26  "status": { "errors": [], "message": "success" }
27 }
```

Step 2. Payment Authentication Setup

Payment Authentication Setup

Description: Submits the `pm_` token against the tracker to trigger direct authorization. No 3DS challenge is performed. The tracker state will transition to `TRACKER_ENDED` on success, indicating the payment is complete. No further steps are required after this call.

Method: `POST`

Endpoint: `/order/payments/v3/{{tracker_token}}`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
{{tracker_token}}	Path	String	M	"track_da499080-..."	Tracker token from Step 1.
payload.payment_method.tokenized_card.token	Body	String	M	"pm_ad4b0519-c700-4c7f-8ed1-e0f06944bcab"	The pm_token to charge.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/order/payments/v3/{{tracker_token}}" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4   --header "Content-Type: application/json" \  
5   --data '{  
6     "payload": {  
7       "payment_method": {  
8         "tokenized_card": {  
9           "token": "pm_ad4b0519-c700-4c7f-8ed1-e0f06944bcab"  
10        }  
11      }  
12    }  
13  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_61414d0f-..."	Tracker token.
data.tracker.state	String	M	"TRACKER_ENDED"	Final state. Payment is complete.
data.tracker.mode	String	M	"payment"	Mode resolved at completion.
data.tracker.entry_mode	String	M	"tms"	Tokenized entry mode confirmed.
data.action.payment_method.token	String	M	"card_f116867e-..."	Internal card token used.
data.action.payment_method.card_type	String	M	"Visa"	Card network.
data.action.payment_method.last_four	String	M	"1111"	Last four card digits.
status.errors	Array	M	[]	Empty on success.

status.message	String	M	"success"	Response status message.
----------------	--------	---	-----------	--------------------------

Sample Response

Success (HTTP 200):

```

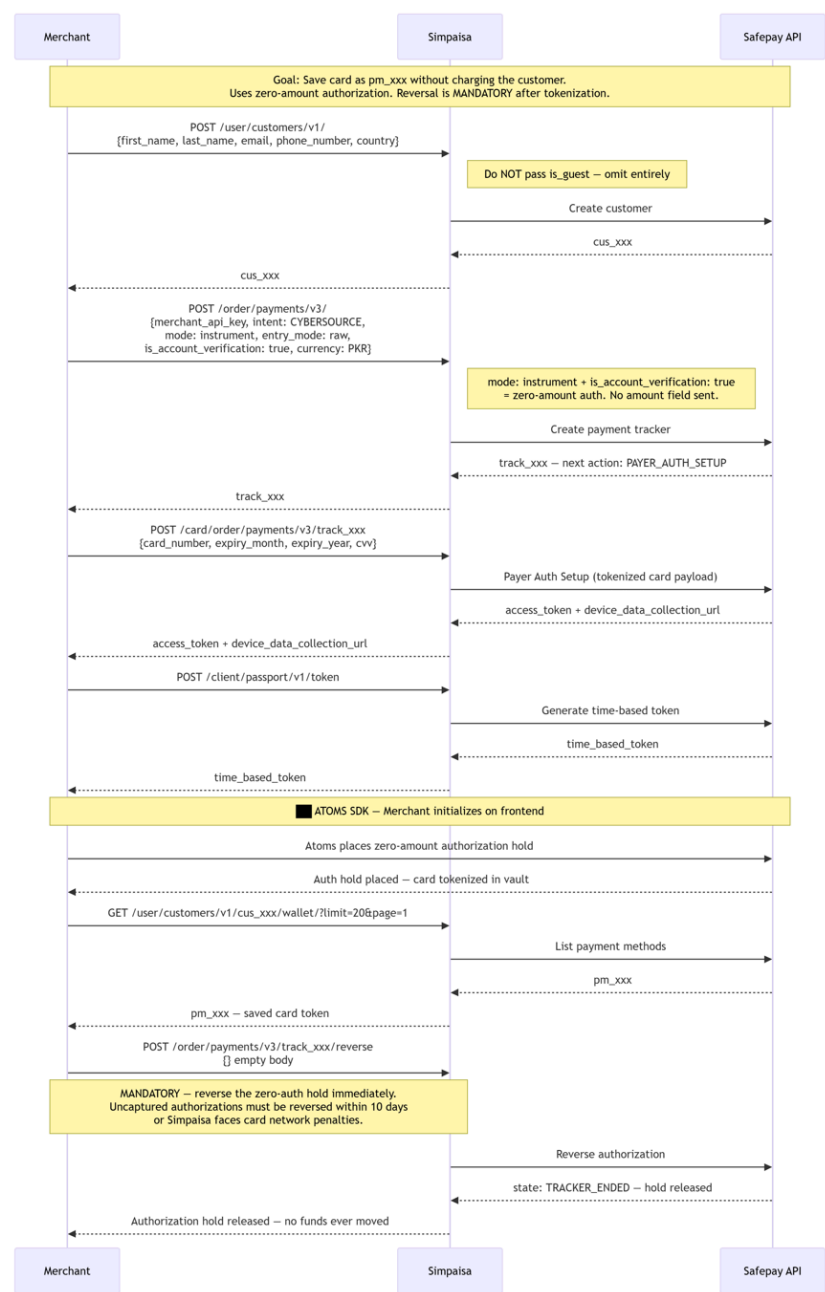
1  {
2    "data": {
3      "tracker": {
4        "token": "track_61414d0f-7740-4297-903a-3aca4b04bca8",
5        "client": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
6        "environment": "sandbox",
7        "state": "TRACKER_ENDED",
8        "intent": "CYBERSOURCE",
9        "mode": "payment",
10       "entry_mode": "tms",
11       "customer": "user_d75110e4-f52e-464d-96bd-490f75e7fc71",
12       "next_actions": {
13         "CYBERSOURCE": { "kind": "PAYER_AUTH_ENROLLMENT",
14           "request_id": "req_ac4307d3-be62-4ea7-ab5f-e950125940df" },
15         "MPGS": { "kind": "NOOP" }
16       },
17       "purchase_totals": {
18         "quote_amount": { "currency": "PKR", "amount": 600000 },
19         "base_amount": { "currency": "PKR", "amount": 600000 },
20         "conversion_rate": { "base_currency": "PKR", "quote_currency":
21           "PKR", "rate": 1 }
22       },
23       "metadata": {}
24     },
25     "action": {
26       "token": "req_ac4307d3-be62-4ea7-ab5f-e950125940df",
27       "payment_method": {
28         "token": "card_f116867e-955a-4d03-802d-1f4d06fe1921",
29         "expiration_month": "12",
30         "expiration_year": "2025",
31         "card_type_code": "001",
32         "card_type": "Visa",
33         "bin_number": "411111",
34         "last_four": "1111"
35       }
36     },
37     "status": { "errors": [], "message": "success" }
38   }

```

7. Flow 4 — Tokenization Without Charge

This flow saves a customer's card without charging them. A zero-amount authorization (`is_account_verification: true`) is used to verify the card and trigger tokenization. After the card is saved and the `pm_` token is retrieved via List Payment Methods, the authorization hold **DOES NOT need to** be reversed using the Reversal API. No capture is ever performed in this flow. I will update diagram below

Sequence Diagram



Step 1. Create Customer

Identical to Flow 1, Step 1. Call `POST /user/customers/v1/` with customer details. Do not pass `is_guest`. The returned `cus_` token is not used in the body of subsequent steps in this flow but is required for List Payment Methods in Step 6.

Step 2. Create Payment Tracker (Instrument Mode, Zero-Auth)

Payment API — Instrument Mode

Description: Creates a tracker specifically for saving a payment method without charging the customer. Set `mode: "instrument"` and `is_account_verification: true` to initiate a zero-amount authorization. Do not include an `amount` field.

Method: POST

Endpoint: `/order/payments/v3/`

Authentication: Header — X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
merchant_api_key	Body	String	M	"sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1"	Merchant API key.
intent	Body	String	M	"CYBERSOURCE"	Payment processor.
mode	Body	String	M	"instrument"	Set to "instrument" for tokenization-only flows.
entry_mode	Body	String	M	"raw"	Card entry mode — raw card details will be submitted.
is_account_verification	Body	Boolean	M	true	Must be true to trigger zero-amount authorization.
currency	Body	String	M	"PKR"	ISO 4217 currency code. Amount field is omitted.

Sample Request (cURL)

```
1 curl --request POST \  
2 --url "{{baseUrl}}/order/payments/v3/" \  
3 --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4 --header "Content-Type: application/json" \  
5 --data '{  
6   "merchant_api_key": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",  
7   "intent": "CYBERSOURCE",  
8   "mode": "instrument",  
9   "entry_mode": "raw",  
10  "is_account_verification": true,  
11  "currency": "PKR"  
12  }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_fb614ed6-718f-414d-906b-4453b5706392"	Tracker token. Pass as {{tracker_token}} in Step 3.
data.tracker.mode	String	M	"instrument"	Confirms instrument mode.
data.tracker.entry_mode	String	M	"raw"	Confirms raw card input.

<code>data.tracker.next_actions.CYBERSOURCE.kind</code>	String	M	"PAYER_AUTH_SETUP"	Next required action — proceed to Step 3.
<code>data.tracker.purchase_totals.quote_amount.currency</code>	String	M	"PKR"	Currency only (no amount for zero-auth).
<code>status.errors</code>	Array	M	[]	Empty on success.
<code>status.message</code>	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1 {
2   "data": {
3     "tracker": {
4       "token": "track_fb614ed6-718f-414d-906b-4453b5706392",
5       "client": "sec_a7cc6fc1-088d-4f35-9dac-2bab2cb234a1",
6       "environment": "sandbox",
7       "state": "TRACKER_STARTED",
8       "payment_method_kind": "card",
9       "intent": "CYBERSOURCE",
10      "mode": "instrument",
11      "entry_mode": "raw",
12      "next_actions": {
13        "CYBERSOURCE": { "kind": "PAYER_AUTH_SETUP" },
14        "MPGS": { "kind": "NOOP" },
15        "PAYFAST": { "kind": "NOOP" }
16      },
17      "purchase_totals": {
18        "quote_amount": { "currency": "PKR" },
19        "base_amount": { "currency": "PKR" },
20        "conversion_rate": { "base_currency": "PKR", "quote_currency":
21        "PKR", "rate": 1 }
22      },
23      "metadata": {}
24    },
25    "capabilities": { "CYBERSOURCE": true, "MPGS": true, "PAYFAST":
26    true }
27    },
28    "status": { "errors": [], "message": "success" }
29  }

```

Step 3. Payer Authentication Setup

Identical to Flow 1, Step 3. Call `POST /card/order/payments/v3/{{tracker_token}}` against `<https://sandbox-securegateway.simpaisa.com>` with the raw card details. Use the tracker token from Step 2. Returns `access_token` and `device_data_collection_url` for Atoms in Step 5.

Step 4. Time-Based Token

Identical to Flow 1, Step 4. Call `POST /client/passport/v1/token`. The returned `data` string is passed as `authToken` in Atoms.

Step 5. Atoms Module (do_capture: false, do_card_on_file: true)

Atoms Module — Zero-Auth Tokenization

Description: Initialize the Atoms Payer Authentication component with `do_capture: false` (no funds are captured) and `do_card_on_file: true` (card must be tokenized). This triggers a zero-amount authorization for card verification and saves the card as a `pm_` token in the customer's wallet.

```

1 <PayerAuthentication
2   environment="sandbox"
3   tracker="{{tracker_token}}"
4   authToken="{{time_based_token}}"
5   deviceDataCollectionJWT="{{access_token}}"
6   deviceDataCollectionURL="{{device_data_collection_url}}"
7   authorizationOptions={{
8     do_capture: false,      // MUST be false - this is a zero-auth
9     only, no capture
10    do_card_on_file: true   // MUST be true - tokenize the card

```

```

10  }}
11  imperativeRef={authRef}
12  onPayerAuthenticationFailure={handleFailure}
13  onPayerAuthenticationSuccess={handleSuccess}
14  onPayerAuthenticationFrictionless={handleSuccess}
15  onPayerAuthenticationUnavailable={handleFailure}
16  onSafepayError={handleFailure}
17 />

```

Prop	Source in This Flow
tracker	Step 2 response — data.tracker.token
authToken	Step 4 response — data
deviceDataCollectionJWT	Step 3 response — data.action.payer_authentication_setup.access_token
deviceDataCollectionURL	Step 3 response — data.action.payer_authentication_setup.device_data_collection_url

Step 6. List Payment Methods

Identical to Flow 1, Step 7. Call `GET /user/customers/v1/{{customer_token}}/wallet/?limit=20&page=1` using the `cus_token` from Step 1. Retrieve the `pm_token` from the response for your records. After this step, proceed immediately to Step 7 (Reversal).

8. Utilities — Reversal & Refund

Use the following decision table to determine the correct API for a given transaction state:

Scenario	API to Use
Auth-only transaction (not yet captured)	Reversal
Auth + capture transaction within 24 hours of capture	Refund
Captured transaction after 24 hours	Refund
Zero-amount authorization from Flow 4 (Tokenization Without Charge)	Not Needed

Critical Rule: Uncaptured authorizations **must** be reversed within **10 days** or Simpaisa faces penalties from card networks. These windows are non-negotiable.

Reversal API

Reversal

Description: Cancels an authorization and releases the hold on the customer's funds. Used for auth-only transactions at any time within 10 days, and is not mandatory for zero-amount authorizations from Flow 4. The request body may be empty. The `{{tracker_token}}` path parameter is the tracker token for the transaction to reverse.

Method: POST

Endpoint: /order/payments/v3/{{tracker_token}}/reverse

Authentication: Header — X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	{{merchant_secret_key}}	Merchant authentication secret key.
Content-Type	Header	String	M	application/json	Request payload format.
{{tracker_token}}	Path	String	M	"track_1a46dc70- ..."	Tracker token of the transaction to reverse.

Sample Request (cURL)

```
1 curl --request POST \  
2   --url "{{baseUrl}}/order/payments/v3/{{tracker_token}}/reverse" \  
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \  
4   --header "Content-Type: application/json" \  
5   --data '{}'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_1a46dc70-91f9-4892-8e25-0bb5205b69c7"	Reversed tracker token.
data.tracker.state	String	M	"TRACKER_ENDED"	Final state post-reversal.
data.tracker.next_actions.CYBERSOURCE.kind	String	M	"N00P"	No further actions required.
data.tracker.next_actions.CYBERSOURCE.request_id	String	M	"req_9ea53be5- ..."	Request ID of last action.
data.tracker.purchase_totals.quote_amount.amount	Number	M	600000	Original authorized amount.
data.tracker.purchase_totals.quote_amount.currency	String	M	"PKR"	Currency.
data.action.token	String	M	"req_9ea53be5- ..."	Action token.
data.action.payment_method.token	String	M	"card_353d4bd9- ..."	Card token used.
data.action.payment_method.card_type	String	M	"Visa"	Card network.

<code>data.action.payment_method.last_four</code>	String	M	"1096"	Last four card digits.
<code>status.errors</code>	Array	M	[]	Empty on success.
<code>status.message</code>	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```
1 {
2   "data": {
3     "tracker": {
4       "token": "track_1a46dc70-91f9-4892-8e25-0bb5205b69c7",
5       "client": "sec_eade52-46aa-483b-b87d-009d3ce41554",
6       "environment": "sandbox",
7       "state": "TRACKER_ENDED",
8       "intent": "CYBERSOURCE",
9       "mode": "payment",
10      "next_actions": {
11        "CYBERSOURCE": { "kind": "N00P", "request_id": "req_9ea53be5-
12          828f-4730-b187-9d3d350ab72a" }
13      },
14      "purchase_totals": {
15        "quote_amount": { "currency": "PKR", "amount": 600000 },
16        "base_amount": { "currency": "PKR", "amount": 600000 },
17        "conversion_rate": { "base_currency": "PKR", "quote_currency":
18          "PKR", "rate": 1 }
19      },
20      "action": {
21        "token": "req_9ea53be5-828f-4730-b187-9d3d350ab72a",
22        "payment_method": {
23          "token": "card_353d4bd9-1d72-4f4d-a358-b277833795a4",
24          "expiration_month": "12",
25          "expiration_year": "2023",
26          "card_type_code": "001",
27          "card_type": "Visa",
28          "bin_number": "445653",
29          "last_four": "1096"
30        }
31      },
32      "status": { "errors": [], "message": "success" }
33    }
```

Refund API

Refund

Description: Returns funds to the customer for a previously captured transaction. Supports partial refunds — if `amount` is provided, only that amount is refunded; if `amount` is omitted, a full refund is issued. Available for up to 90 days after the original transaction.

The `{{tracker_token}}` path parameter is the tracker token of the original captured transaction.

Method: POST

Endpoint: `/order/payments/v3/{{tracker_token}}/refund`

Authentication: Header — `X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}`

Request Parameters

M = Mandatory | O = Optional | C = Conditional

Parameter	Location	Data Type	M/O/C	Sample Value	Description
X-SFPY-MERCHANT-SECRET	Header	String	M	<code>{{merchant_secret_key}}</code>	Merchant authentication secret key.

Content-Type	Header	String	M	application/json	Request payload format.
{{tracker_token}}	Path	String	M	"track_13e12198-..."	Tracker token of the captured transaction to refund.
currency	Body	String	M	"PKR"	ISO 4217 currency code of the refund amount.
amount	Body	Number	O	600000	Amount to refund in smallest currency unit. If omitted, the full transaction amount is refunded. Partial refunds are supported and unlimited in count.

Sample Request (cURL)

```

1 curl --request POST \
2   --url "{{baseUrl}}/order/payments/v3/{{tracker_token}}/refund" \
3   --header "X-SFPY-MERCHANT-SECRET: {{merchant_secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{
6     "currency": "PKR",
7     "amount": 600000
8   }'
```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
data.tracker.token	String	M	"track_13e12198-8fc4-435f-8dc0-2fd3390585e8"	Tracker token of the refunded transaction.
data.tracker.state	String	M	"TRACKER_REFUNDED"	TRACKER_REFUNDED for full refund; TRACKER_PARTIAL_REFUND for partial refund.
data.tracker.mode	String	M	"payment"	Original transaction mode.
data.tracker.entry_mode	String	M	"unscheduled_raw"	Original card entry mode.
data.tracker.customer	String	M	"cus_f27180ce-..."	Customer token.
data.tracker.next_actions. CYBERSOURCE.kind	String	M	"N00P"	No further actions.
data.tracker.purchase_total. quote_amount.amount	Number	M	600000	Original transaction amount.

<code>data.tracker.purchase_totals.quote_amount.currency</code>	String	M	"PKR"	Currency.
<code>data.tracker.authorization.token</code>	String	M	"auth_a8d40706-4218-432d-987e-03f26601008c"	Authorization token of the original transaction.
<code>data.tracker.authorization.tracker</code>	String	M	"track_13e12198-..."	Tracker linked to the authorization.
<code>data.tracker.authorization.attempt</code>	String	M	"attemp_4d4b205c-..."	Attempt ID.
<code>data.tracker.authorization.cybersource_rid</code>	String	M	"7636281971636139904807"	CyberSource reference ID.
<code>data.tracker.authorization.totals.amount</code>	Number	M	600000	Original authorized amount.
<code>data.tracker.authorization.totals.currency</code>	String	M	"PKR"	Currency.
<code>data.tracker.authorization.created_at.seconds</code>	Number	M	1763628199	Unix timestamp of original authorization.
<code>data.tracker.authorization.updated_at.seconds</code>	Number	M	1763628199	Unix timestamp of latest update (refund).
<code>data.action</code>	Object	M	{}	Empty — refund has no next actions.
<code>status.errors</code>	Array	M	[]	Empty on success.
<code>status.message</code>	String	M	"success"	Response status message.

Sample Response

Success (HTTP 200):

```

1  {
2    "data": {
3      "tracker": {
4        "token": "track_13e12198-8fc4-435f-8dc0-2fd3390585e8",
5        "client": "sec_97d12293-91de-4a0e-ba1b-099bc4e4bd19",
6        "environment": "sandbox",
7        "state": "TRACKER_REFUNDED",
8        "payment_method_kind": "card",
9        "intent": "CYBERSOURCE",
10       "mode": "payment",
11       "entry_mode": "unscheduled_raw",
12       "customer": "cus_f27180ce-d143-4480-9aae-d47fdd3650ba",
13       "next_actions": {
14         "CYBERSOURCE": { "kind": "N00P", "request_id": "req_34ef4975-
2ade-44de-87e9-743d35b4e703" },
15         "MPGS": { "kind": "N00P" },
16         "PAYFAST": { "kind": "N00P" }
17       },
18       "purchase_totals": {
19         "quote_amount": { "currency": "PKR", "amount": 600000 },
20         "base_amount": { "currency": "PKR", "amount": 600000 },
21         "conversion_rate": { "base_currency": "PKR", "quote_currency":
"PKR", "rate": 1 }
22       },
23       "metadata": {},
24       "authorization": {
25         "token": "auth_a8d40706-4218-432d-987e-03f26601008c",
26         "tracker": "track_13e12198-8fc4-435f-8dc0-2fd3390585e8",
27         "attempt": "attemp_4d4b205c-5beb-4ee5-b680-7b106569600b",
28         "intent": "CYBERSOURCE",
29         "cybersource_rid": "7636281971636139904807",
30         "totals": { "currency": "PKR", "amount": 600000 },
31         "created_at": { "seconds": 1763628199 },
32         "updated_at": { "seconds": 1763628199 }
33       }
34     },
35     "action": {}

```

```
36 },  
37 "status": { "errors": [], "message": "success" }  
38 }
```